
سوال 1)

تمام فایل هایی که تغییر دادم را ضمیمه کردم . همچنین عکس خروجی فایل تست را نیز ضمیمه کردم .
ابتدا فایل umalloc را توضیح میدهم :

1- تابع realloc :

این تابع سه حالت دارد :

حالت اول : اگر $ptr == NULL$ باشد اون وقت رفتار مشابه $malloc(nbytes)$ باید داشته باشد :

```
if(ptr == 0){  
    return malloc(nbytes);  
}
```

حالت دوم : اگر $nbytes == 0$ باشد اون وقت باید مثل تابع $free(ptr)$ عمل کنه و مقدار $NULL$ رو برگردونه :

```
if(nbytes == 0){  
    free(ptr);  
    return 0;  
}
```

حالت سوم : اگر جز سه حالت بالا نباشه اون وقت در ابتدا باید پیام ptr رو به نوع $Header*$ تبدیل میکنم و سپس یک واحد میرم عقب تا به $header$ بلوک فعلی حافظه برسم :

```
Header *bp = (Header*)ptr - 1;
```

سپس مقدار حافظه ای که قبلا برای داده ها اختصاص داده شده بود رو محاسبه میکنم :

```
uint oldbytes = (bp->s.size - 1) * sizeof(Header);
```

سپس یک بلوک حافظه جدید با اندازه خواسته شده میسازم :

```
void *newp = malloc(nbytes);
```

سپس آگه تخصیص حافظه جدید ناموفق باشد تابع متوقف می‌شود :

```
if(newp == 0)
    return 0;
```

حالا تعداد بایت هایی که باید از حافظه قبلی کپی بشود رو تعیین می‌کنم :

```
uint minbytes = oldbytes < nbytes ? oldbytes : nbytes;
```

سپس داده های قبلی رو به حافظه جدید کپی می‌کنم :

```
memmove(newp, ptr, minbytes);
```

و در نهایت بلوک حافظه قبلی رو آزاد می‌کنم و آدرس بلوک حافظه جدید رو return می‌کنم :

```
free(ptr);
return newp;
```

2- تابع calloc :

در ابتدا بررسی می‌کنم هنگام محاسبه num*size سرریز داره یا نه :

```
if(size != 0 && num > 0xFFFFFFFFu / size)
    return 0;
```

سپس مقدار کل حافظه رو به بایت حساب می‌کنم :

```
uint total = num * size;
```

سپس حافظه ای با اندازه total می‌سازم :

```
void *p = malloc(total);
```

آگه تخصیص تابع موفق نبود تابع متوقف می‌شود :

```
if(p == 0)
    return 0;
```

در انتها تمام بایت های حافظه رو با صفر مقداردهی می‌کنم :

```
memset(p, 0, total);
return p;
```

3- فایل test.c :

a همان pointer اصلی به حافظه پویا است. b هم برای نگه داشتن آدرس قبلی تعریف کردم برای تست free کردن حافظه. i هم که شمارنده حلقه هام هست.

1- تست calloc :

در ابتدا برای 5 عدد int دستور calloc رو زدم و سپس مقادیر آرایه رو چاپ میکنم :

```
printf("--calloc test--\n");
a = (int*)calloc(5, sizeof(int));

for(i = 0; i < 5; i++)
    printf("a[%d]= %d\n", i, a[i]);
```

2- تست realloc :

حالت اول : بزرگتر کردن حافظه ← در اینجا حافظه رو از 5 تا 10 تبدیل به 10 تا int میکنم و باید دقت کنم که داده های قبلی با این دستور حذف نمیشوند. سپس عناصر جدید رو مقدار دهی کردم تا ببینم درست کار میکنه و در نهایت تمام عناصر رو چاپ میکنم :

```
printf("\n--realloc (expand) test--\n");
printf("increase memmory\n");
a = realloc(a, 10 * sizeof(int));

for(i = 5; i < 10; i++)
    a[i] = i * 7;

for(i = 0; i < 10; i++)
    printf("a[%d]= %d\n", i, a[i]);
```

کوچک کردن حافظه ← در اینجا حافظه رو از 10 تا 2 تبدیل به 2 تا int میکنم و باید دقت کنم که داده های قبلی با این دستور حذف نمیشوند. در نهایت تمام عناصر رو چاپ میکنم :

```
printf("decrease memmory\n");
a = realloc(a, 2 * sizeof(int));

for(i = 0; i < 2; i++)
    printf("a[%d]= %d\n", i, a[i]);
```

حالت دوم: $ptr = 0$ ← در این حالت باید اول آدرس فعلی a رو داخل b میزارم برای مقایسه بعدی. سپس درخواست $realloc$ با اندازه صفر باید مثل $free(a)$ عمل کنه و $NULL$ برگردونه و سپس بررسی میکنم آیا $NULL$ برگردونده یا نه:

```
printf("\n--realloc(ptr, 0) test--\n");
b = a;
a = realloc(a, 0);

if(a == 0)
    printf("realloc(ptr, 0) must return NULL\n");
```

حالا میام دوباره یک حافظه به اندازه 10 میگیرم تا ببینم آدرس قبلی آزاد شده و امکان استفاده مجدد هست یا نه. حالا اگر $a=b$ یعنی همان فضای آزاد شده دوباره استفاده شده. اگر هم $a \neq b$ اون وقت یعنی جای دیگه ای تخصیص داده شده. در انتها هم حافظه ای که با $malloc$ گرفته بودم رو آزاد میکنم:

```
a = malloc(10 * sizeof(int));
if(a == b)
    printf("free and refuse memory\n");
else
    printf("memory freed and address changed\n");

free(a);
```

حالت سوم: $realloc(NULL, size)$ ← این حالت باید مثل $malloc(size)$ عمل کنه و حافظه بده بنابراین در ابتدا چک میکنم که a مخالف صفر است یا نه تا بفهمم واقعا حافظه تخصیص داده شده یا نه سپس مقادیر آرایه رو چاپ میکنم باید به این دقت کنیم که مقادیر اولیه $malloc$ مشخص نیست به عبارتی لزوما این مقادیر صفر نیستند. سپس مقادیر آرایه رو دستی مقداردهی کردم و سپس مقدار هارو چاپ کردم تا از درست بودن حافظه مطمئن بشم:

```
printf("\n--(NULL, size) test--\n");
a = realloc(0, 4 * sizeof(int));

if(a != 0)
    printf("1-realloc(NULL, size) behaves like malloc\n");

printf("2-Initial values with undefined value so behaves like
malloc) : \n");
for(i = 0; i < 4; i++)
    printf("a[%d]= %d\n", i, a[i]);
```

```
printf("3-values after manual initialization(i+2) : \n");
for(i = 0; i < 4; i++)
    a[i] = i + 2;

for(i = 0; i < 4; i++)
    printf("a[%d]= %d\n", i, a[i]);
```

در نهایت هم حافظه نهایی رو آزاد کردم و exit کردم .

4- فایل user.h :

این سه خط رو به system calls اضافه کردم :

```
void free(void*);
void* realloc(void *ptr, uint nbytes);
void* calloc(uint num, uint size);
```

5- فایل makefile :

خط زیر رو در UPROGS در makefile اضافه کردم :

```
$U/_test\
```

تحلیل خروجی تست :

پس از اجرای test خروجی در ترمینال به صورت زیر میشود :

```
$ qemu-system-riscv64 --help
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -march=rv64gc -MD -mmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -fno-builtin-stack-protector -fno-pie -no-pie -c -o user/test.o user/test.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/test user/test.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/test > user/test.asm
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_ustest
s user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_LT user/_test
mmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
ballocc: first 1040 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ test
--calloc test--
a[0]= 0
a[1]= 0
a[2]= 0
a[3]= 0
a[4]= 0

--realloc (expand) test--
increase memory
a[0]= 0
a[1]= 0
a[2]= 0
a[3]= 0
a[4]= 0
a[5]= 35
a[6]= 42
a[7]= 49
a[8]= 56
a[9]= 63
decrease memory
a[0]= 0
a[1]= 0

--realloc(ptr, 0) test--
realloc(ptr, 0) returned NULL
memory freed and address changed

--(NULL, size) test--
1-realloc(NULL, size) behaves like malloc
2-initial values with undefined value so behaves like malloc :
a[0]= 0
a[1]= 0
a[2]= 0
a[3]= 0
3-values after manual initialization(i+2) :
a[0]= 2
a[1]= 3
a[2]= 4
a[3]= 5
$
```

برای تست `calloc` که 5 تا حافظه تخصیص دادم و در نتیجه به من 5 تا حافظه داده و مقدار اولیون هم صفر هست.

برای تست `realloc` در حالت `increase` اول اندازه حافظه رو به 10 افزایش دادم و `a[5]` تا `a[9]` رو مقداردهی کردم و همون طور که میبینید این مقادیر درست مقداردهی شدند و همچنین حافظه قبلی هم مقادیر خودش رو حفظ کرده . حالا در حالت `decrease` هم اندازه حافظه رو از 10 تبدیل کردم به 2 و همون طور که میبینید این خونه ها مقدار قبلی خودشون رو حفظ کرد .

برای تست `realloc(ptr,0)` که همان `nbyte=0` است هم همون طور که میبینید مقدار بازگشتی `NULL` میشود(اگر مقدار بازگشتی تابع `NULL` باشه این خط چاپ میشه) و حافظه قبلی آزاد میشه و تخصیص بعدی مقداری متفاوت دارد .

برای تست `realloc(NULL,size)` که همان `ptr=0` است هم همون طور که میبینید حافظه با موفقیت تخصیص داده شده است (رفتار مشابه `malloc`)